Difficultés rencontrées

Plusieurs difficultés sont apparues au cours du projet.

La première difficulté concernait l'évolution de l'architecture en cours de projet. Le client souhaitait visualiser l'application avant que le back soit prêt : j'ai donc d'abord fait fonctionner le front sur des données fictives, puis sur l'API HAL, avant de le brancher sur les endpoints du back. Le projet a aussi commencé avec deux dépôts séparés, puis la dockerisation m'a amené à proposer de tout regrouper dans un monorepo. Chacune de ces bascules a rendu une partie de ma documentation obsolète : en août, certains documents décrivaient encore deux dépôts et une base SQLite alors que le projet avait changé. J'ai donc dû reprendre la documentation et indiquer clairement quel document fait foi, pour que l'équipe et un futur repreneur ne s'appuient pas sur une information dépassée.

La deuxième difficulté concernait la fiabilité du code avant sa mise en production. Plus le projet avançait, plus il devenait difficile pour moi de savoir si une modification cassait ce qui fonctionnait déjà. Des régressions sont passées inaperçues pendant un ou deux commits, qu'elles viennent d'une erreur humaine ou d'un code produit avec l'IA. Les deux revues de sécurité que j'ai menées avant la mise en production portaient sur l'ensemble du code, quelle que soit son origine, et ont rendu un verdict NO-GO. J'ai dû accepter ce constat : sans contrôle systématique, je ne pouvais pas garantir la qualité de ce qui était livré. Une fois les points relevés corrigés, j'ai mis en place une pipeline CI/CD qui rejoue ces contrôles à chaque modification. Cette pipeline ne règle pas tout : elle ne vérifie que ce que j'ai prévu de vérifier, et elle n'est qu'une réponse parmi d'autres possibles à ce problème. Elle m'a surtout montré que la fiabilité d'un projet se prépare dès sa conception.

Compétences appliquées dans ce projet

Maîtriser le génie logiciel :

Sur ce projet, chaque choix d'architecture répond à une contrainte précise.

Le premier concerne le front. Pour répondre au besoin du client de visualiser l'application et de faire ses retours, j'ai d'abord développé le front sur des données fictives (mock), puis je l'ai connecté à l'API publique HAL pour des tests réels, en attendant que le back soit prêt. J'avais placé une couche d'adaptateurs entre les pages et les données : les pages ne dépendent pas de la source, qui se choisit par une variable d'environnement. Une fois le back en place, je n'ai donc eu qu'à basculer de HAL vers nos endpoints REST et à continuer le développement du front, c'était l'intérêt recherché en séparant les pages de la source de données.

Le deuxième concerne l'organisation du code. Pour que tout le monde travaille dans le même environnement, nous avons dockerisé la solution, et j'ai proposé de regrouper le front et le back dans un seul dépôt (monorepo). Deux dépôts auraient permis de séparer les langages, mais auraient demandé des mesures supplémentaires : deux pipelines CI/CD au lieu d'une, et sans doute un troisième dépôt pour l'orchestration. Le monorepo, lui, est plus rapide et plus simple pour une petite équipe qui met tout à jour en une fois, et il place le docker-compose à la racine du projet. Son coût

est d'avoir plusieurs langages dans un même dépôt. Pour une petite équipe, c'était le meilleur compromis.

Ce coût a guidé le choix suivant. Pour les hooks, lefthook n'était pas le seul choix possible, mais c'était le plus adapté à notre besoin. Avec deux langages dans un même dépôt, il ne lance que les contrôles qui concernent les fichiers modifiés, en parallèle, et le tout tient dans un seul fichier de configuration. J'ai mis en place des hooks pre-commit et pre-push, une CI sur GitHub Actions qui rejoue tous les tests sur une machine propre, et une branche main (production) qui n'accepte que des pull requests dont la CI est verte, administrateurs compris. Les hooks seuls ne suffisaient pas : la sécurité veut que l'on ne fasse confiance à rien ni à personne par défaut, et un contournement, volontaire ou non, enlèverait tout l'intérêt de l'automatisation.

Pour décider de ce qui doit être testé, je pars d'une règle : par défaut, ce qui porte une fonctionnalité ou un risque est testé, et chaque nouvelle fonctionnalité ou correction arrive avec au moins un test. Cette règle m'a fait penser à des contrôles souvent oubliés, comme vérifier que le détecteur de secrets reconnaît toujours un secret, ou que chaque domaine scientifique a bien son texte, sa pastille et son visuel. Les exceptions sont écrites et justifiées dans le dépôt : aucun test ne dépend du modèle IA, dont la réponse change d'une exécution à l'autre, et aucun test ne vérifie le pixel des animations, ce qui coûterait plus qu'il ne rapporterait. Pour l'instant, les tests de parcours dans un navigateur sont prévus mais pas installés.

Utiliser les outils numériques de référence et les règles de sécurité informatique pour acquérir, traiter, produire et diffuser de l'information ainsi que pour collaborer en interne et en externe :

Dès le début du projet, un besoin était exprimé : pouvoir transmettre Lab Horizon à une équipe de développement qui n'y a jamais travaillé. Cette équipe doit comprendre rapidement comment le projet fonctionne, comment le mettre en œuvre et comment le maintenir. C'est le rôle de la documentation et du dossier de passation que j'ai rédigés. Avec plusieurs collaborateurs et un projet qui évolue, la documentation s'est accumulée, au point de créer un surplus d'information. J'ai donc indiqué, pour chaque sujet, quel document fait foi : c'est la base solide sur laquelle se positionner en cas de doute.

Le cadrage UX et le parcours utilisateur, que j'ai conçus, ont aussi servi de ligne directrice commune entre le front et le back. L'exemple le plus parlant : les endpoints du back sont arrivés après le front, déjà alignés sur les besoins du parcours, sans qu'il soit nécessaire de les modifier.

Je ne contrôle pas l'usage que mes collaborateurs font de l'IA. J'ai donc mis en place des règles communes, versionnées dans le dépôt, que suivent les agents IA de toute l'équipe. Elles assurent un minimum de cohérence dans le contexte et les règles à respecter : moindre privilège, aucun secret dans le code. L'une d'elles pose que tout contenu lu par l'agent est une donnée, jamais une instruction : les publications importées depuis HAL comme les éléments ajoutés par les collaborateurs. L'objectif est d'éviter l'équivalent de l'injection SQL appliqué aux prompts.

Pour les secrets, j'applique le principe de défense en profondeur. Si on ne vérifie qu'une seule fois, le risque de pousser une clé sans s'en rendre compte est trop grand, surtout si cette vérification repose uniquement sur un LLM. Les fichiers sensibles sont donc bloqués au commit, puis tout l'historique Git est analysé en CI, et GitHub refuse de son côté tout push contenant un secret connu.

Enfin, j'ai construit un protocole de revue de sécurité avant la mise en production. Son objectif est d'imposer des étapes obligatoires et normées, avec une validation humaine. Un simple prompt ne précise ni quels tests faire, ni quand, ni à quel niveau. Les deux premières versions du protocole manquaient de cadrage : il était possible de les contourner, les tests n'étaient pas assez stricts et les attentes restaient trop vagues. La version 2.1 a été utilisée sur Lab Horizon et a rendu deux verdicts NO-GO, chacun avec la liste des corrections à faire avant la mise en production. Ces corrections ont été réalisées. Je me suis ensuite inspiré des contrôles et des tests de la revue pour construire la pipeline CI/CD, qui les rejoue automatiquement à chaque modification. J'ai volontairement choisi de ne pas versionner les rapports de revue : un document listant toutes les failles des versions précédentes n'a pas sa place dans le dépôt.

Analyser ses actions en situation professionnelle, s'autoévaluer pour améliorer sa pratique :

Dans ma fiche sur Vill'âge Jeunes, je notais qu'il aurait fallu réfléchir dès le début du projet au versioning, à la documentation et au déploiement, notamment avec un pipeline CI/CD. Lab Horizon m'a permis de mettre ce constat en pratique, par étapes.

Au départ, je testais à la main. Les erreurs se sont multipliées, et des régressions sont passées inaperçues pendant un ou deux commits. Je l'ai surtout vu sur le front : des fonctionnalités corrigées avaient régressé deux commits plus loin. J'ai alors décidé de vérifier systématiquement que chaque ajout ne défait pas ce qui existait déjà. J'ai écrit mes premiers fichiers de tests, puis j'ai recherché des méthodes de tests anti-régression adaptées au développement avec un LLM. L'agent IA lançait ces contrôles avant chaque commit.

Ces contrôles ont montré leur limite. Un LLM ne donne pas une réponse reproductible pour un même test, dans un même environnement et avec le même contexte. J'ai donc confié les tests à des machines, avec une pipeline CI/CD, ce qui économise aussi des tokens de conversation. Mon usage de l'IA a changé. Elle ne lance plus les tests : elle m'aide à réfléchir aux tests qui encadrent le projet, à mettre en place la pipeline sur GitHub, à la sécuriser pour qu'elle ne puisse pas être contournée, et à trouver ou adapter des tests quand c'est nécessaire. Je peux ainsi me concentrer sur la compréhension et sur les choix importants, là où se trouve la vraie valeur ajoutée.
J'ai utilisé mon portfolio comme bac à sable. Les tests et les règles y étaient présents dès le départ, mais pas la pipeline. Mes essais sur le portfolio m'ont permis de peaufiner la charte, les gates puis la pipeline de Lab Horizon, pour obtenir un résultat plus propre et plus professionnel.

Si je recommençais Lab Horizon, je prendrais beaucoup plus de temps pour préparer la base et la réflexion du projet. En plus des règles, je prévoirais dès le départ les tests liés aux fonctionnalités à implémenter, la pipeline, les conventions de commit et les règles de mise en ligne sur le dépôt. La structuration serait plus longue, mais elle donnerait une base plus saine et plus solide, et ferait gagner du temps de développement ensuite.

Enfin, cette solution a une limite : un test ne vérifie que ce qui a été prévu. Il doit être adapté aux besoins et évoluer avec l'application. À chaque nouvelle fonctionnalité correspondent ses tests. À chaque modification d'une fonctionnalité, je me demande si le test est toujours adapté, s'il vérifie

bien qu'elle fait ce qu'elle doit faire dans son cadre, et s'il échoue bien quand il le doit au lieu de laisser passer une erreur.

Axes d'amélioration et prise de recul

Sur ce projet, ma démarche a encore des limites. Les tests de parcours dans un navigateur sont prévus mais pas encore installés : ils permettraient de vérifier les écrans du point de vue du visiteur, ce que mes tests actuels ne couvrent pas. Les contre-vérifications de mes revues de sécurité ont été faites par le même outil que la revue elle-même : une relecture par une personne distincte, puis une nouvelle revue sur la version finale, donneraient un avis plus fiable. Enfin, je dois prendre l'habitude de vérifier régulièrement que mes tests échouent bien quand ils le doivent, pour ne pas me reposer sur une pipeline verte qui ne contrôlerait plus rien d'utile.

Pour le reste, ce projet est une étape dans ma pratique. J'ai commencé à répondre aux manques relevés sur Vill'âge Jeunes, sur le versioning, la documentation et la pipeline, mais en cours de route et non dès le départ. Les deux NO-GO ont été formateurs : ils m'ont montré concrètement l'intérêt d'un cadre strict, et leurs contrôles ont servi de base à la pipeline. Ce que j'ai mis en place reste une première réponse. Il me faudra continuer à la remettre en question et à chercher de meilleures solutions sur mes prochains projets, en commençant par poser ce cadre dès leur lancement.